

enumerate_li_configs_Final_optimized_motif_v6_mtp_train — 사용 매뉴얼

대상 파일:

enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py

버전:

v6 (MTP 에너지 백엔드 + 모티프 시딩 GA + 학습 데이터 생성 + 구조 variant)

기반 논문:

Kim et al., ACS Nano — "Revealing Li Staging Reactions in Graphite via a GA Coupled with MLIP"

목차

- 1. 개요
- 2. 요구 사항
- 3. 입력 파일
- 4. 출력 디렉토리 구조
- 5. 전체 옵션 레퍼런스
- 6. 사용 예시
- 7. 워크플로우 가이드
- 8. 성능 튜닝 가이드
- 9. 출력 파일 설명
- 10. 자주 묻는 질문

1. 개요

흑연 삽입 화합물(GIC, Graphite Intercalation Compound)에서 Li-vacancy 배열을 열거하고, 각 SOC(State of Charge) 레벨에서 가장 안정한 구조를 탐색하는 스크립트입니다.

동작 모드

모드	설명	사용 시기
전수 열거 (full enumeration)	대칭 유일 대표 구조를 모두 생성	소형 슈퍼셀, Li 수가 적을 때
GA (Genetic Algorithm)	유전 알고리즘으로 광대한 공간 탐색	대형 슈퍼셀, Li 수가 많을 때

모드	설명	사용 시기
MTP 학습 데이터 생성 ★ NEW	전수열거 + GA supplement + Variant 구조 → CFG 파일	MLIP 포텐셜 학습 데이터 구축

에너지 백엔드

백엔드	설명	기본값
<code>mtp-relax</code>	<code>mlp relax</code> 로 MLIP 구조 완화 후 에너지	✓ 기본
<code>ewald</code>	Ewald 전기정적 에너지 (빠르나 근사)	학습 데이터 생성 권장

2. 요구 사항

```
# Python 3.8+
pip install pymatgen numpy matplotlib tqdm

# MLIP 실행파일 (mtp-relax 백엔드 사용 시)
# mlp 바이너리가 PATH에 있어야 함
mlp --version

# 필수 파일
# - 구조 파일: LiC6.cif 또는 POSCAR
# - (mtp-relax 시) mlp.ini: MLIP 포텐셜 설정 파일
```

3. 입력 파일

구조 파일 (`--structure`)

LiC6 또는 LiC12 등 GIC 단위셀 CIF 또는 POSCAR

Li와 C 원소가 모두 포함되어야 함

슈퍼셀 확장 전 단위셀 형태로 제공

mlp.ini (`--mlip-ini`)

MLIP 포텐셜 파라미터 파일

`mlp relax` 실행에 필요

`mtp-relax` 백엔드 사용 시 필수

4. 출력 디렉토리 구조

일반 모드 (전수 열거 / GA)

```
li_configs/                                     ← --output (기본: ./li_configs)
├──
├── enumeration_summary.json                     ← 전체 실행 요약 (SOC별 최안정 구조, 통계)
├──
├── SOC_0.5000_nLi08/                           ← SOC=0.5, n_Li=8 인 경우
│   ├── top_stable_cif/
│   │   ├── rank001_AA_nLi08_SOC0.5000_E-14.777eV.cif
│   │   └── ... (최대 20개)
│   └── ga_convergence_nLi08.png                ← GA 수렴 그래프 (--ga 시)
```

학습 데이터 생성 모드 (`--write-training-cfg`) ★ NEW

```

li_configs/
|
|—— enumeration_summary.json
|—— training_data.cfg          ← 전체 학습 구조 통합 CFG (MTP 학습 입력)
|
|—— SOC_0.5000_nLi08/         ← 일반 출력 (위와 동일)
|   |—— top_stable_cif/
|
|—— training_data/            ← 학습 데이터 루트
|   |—— SOC_0.0000_nLi00/
|       |—— full_enum/         ← 전수열거 대칭유일 구조 CIF
|           |—— rank00001_AA_nLi00_SOC0.0000_E0.0000eV.cif
|
|   |—— SOC_0.1250_nLi01/
|       |—— full_enum/         ← 원래 Li site 대칭유일 구조
|       |—— ga_supplement/     ← expanded hollow site GA 탐색 구조
|       |—— variant_structures/ ← 설계된 Variant 구조 (6-15번)
|
|   |—— SOC_0.5000_nLi04/
|       |—— full_enum/
|       |—— ga_supplement/     ← --training-data-soc-threshold 1.01 시 전 SOC
|       |—— variant_structures/
|
|   |—— SOC_1.0000_nLi08/
|       |—— full_enum/
|       |—— ga_supplement/
|       |—— variant_structures/

```

학습 데이터 3종 조합 전략:

`full_enum/` : 원래 Li site (N_{LI}_TOTAL 사이트) 에서 대칭 유일 구조, 에너지 오름차순 top-N

`ga_supplement/` : expanded hollow site (흑연 헥사곤 중심 포함)에서 GA 탐색, top-N sym-unique

`variant_structures/` : 10가지 설계 원칙으로 생성된 체계적 구조 (아래 참조)

Cross-source dedup: 세 소스 구조를 합산한 뒤, 소스 우선순위(`full_enum` → `variant_structures` → `ga_supplement`) 순으로 pymatgen StructureMatcher 비교를 수행하여 원자 위치가 같은 중복 구조를 제거합니다. `full_enum` 과 `variant_structures` 가 먼저 확정되고, `ga_supplement` 는 이 두 소스에 없는 hollow-center 신규 구조만 추가합니다.

5. 전체 옵션 레퍼런스

- = 기본값 있음 (생략 가능)
- = 필수 또는 기본값 없음
- ✓ = 기본 활성화된 플래그
- ★ = v6 신규

5.1 기본 설정

옵션	타입	기본값	설명
<code>--structure</code> / <code>-s</code> ●	str	—	단위셀 CIF 또는 POSCAR 파일 경로
<code>--output</code> / <code>-o</code> ●	str	<code>./li_configs</code>	출력 디렉토리 루트
<code>--supercell</code> NX NY NZ ●	int×3	<code>2 2 2</code>	슈퍼셀 확장 비율 (a, b, c 방향)

5.2 열거 모드

옵션	타입	기본값	설명
<code>--no-symmetry</code> ●	flag	off	대칭 비교를 전수 열거 (C(N,k) 전체)
<code>--symprec</code> ●	float	<code>0.01</code> Å	SpacegroupAnalyzer 대칭 정밀도
<code>--match-tol</code> ●	float	<code>0.01</code>	Li 사이트 매칭 분수좌표 허용오차
<code>--soc-levels</code> N [N ...] ●	int+	모든 0...N_Li	처리할 Li 수 목록 (예: <code>--soc-levels 4 8 12</code>)

5.3 GA (유전 알고리즘) 설정

옵션	타입	기본값	설명
<code>--ga</code> ●	flag	off	GA 활성화 (미지정 시 전수 열거)
<code>--ga-pop</code> ●	int	<code>120</code>	세대당 population 크기
<code>--ga-gens</code> ●	int	<code>60</code>	총 세대 수
<code>--ga-threshold-ev</code> ●	float	<code>0.5</code> eV	생존자 선택 에너지 임계값
<code>--ga-seed</code> ●	int	<code>42</code>	난수 시드 (재현성)
<code>--stacking-mutation-prob</code> ●	float	<code>0.15</code>	스태킹 돌연변이 비율 (새 개체 중)

옵션	타입	기본값	설명
<code>--ga-min-li-li-same-layer</code> 	float	4.00 Å	같은 Li 레이어 내 최소 Li-Li 거리

GA 동작 원리: 생존자와 변형

각 세대에서 에너지 평가 → 생존자 선택 → 다음 세대 생성 순으로 진행됩니다.

생존자 선택 (`--ga-threshold-ev`)

`min_E + threshold` 이내의 구조만 생존합니다. 생존자들이 다음 세대의 부모 풀(parent pool)을 구성합니다.

다음 세대 생성 방식 (새 개체 중 비율)

유형	대상	설명
<code>vacancy_swap</code>	생존자	같은 레이어 내 Li↔vacancy 교환
<code>layer_swap</code>	생존자	다른 레이어 간 Li 위치 이동
<code>stacking_flip</code>	생존자	스태킹 시퀀스(AA/AB 등) 변경
<code>elite_perturb</code>	최우수 생존자	1-2 사이트만 교체한 미세 섭동
<code>random</code>	—	부모와 무관한 완전 새 개체 (~25%)
<code>fullwipe</code>	—	전체 Li 재배치 (정체 탈출용, ~5%)

핵심: `vacancy_swap`, `layer_swap`, `stacking_flip`, `elite_perturb` 는 생존자를 부모로 삼아 변형하므로, 좋은 구조 특징을 유지하면서 탐색합니다. `random` 과 `fullwipe` 는 새 개체를 독립 생성하여 다양성을 보충합니다.

`--ga-threshold-ev` 설정 지침

값	선택 압력	추천 상황
0.3 eV	강함 (상위 구조만 생존)	대형 슈퍼셀, Li 수가 많아 수렴이 느릴 때
0.5 eV	보통  기본값	대부분의 경우
0.7 - 1.0 eV	약함 (다수 구조 생존)	소형 슈퍼셀, 초기 다양성 확보가 중요할 때

값이 너무 작으면 생존자가 극소수라 다양성 부족 → 조기 수렴 위험. 값이 너무 크면 선택 압력이 약해 수렴이 느립니다.

5.4 스택킹 시퀀스 설정

옵션	타입	기본값	설명
<code>--stacking-sequences SEQ [SEQ ...]</code> ●	str+	단층(AA 등)	탐색할 C 레이어 스택킹 시퀀스. <code>ALL</code> 로 모든 A/B 조합 사용
<code>--ab-shift DA DB</code> ●	float×2	자동감지	B 레이어 분수좌표 면내 변위 (미지정 시 자동)
<code>--carbon-layer-tol</code> ●	float	0.05	탄소 레이어 그룹핑 분수 z 허용오차

SOC 기반 스택킹 정책

옵션	타입	기본값	설명
<code>--soc-stacking-policy</code> ●	choice	conservative	SOC에 따른 스택킹 공간 축소 정책
<code>--aa-only-soc-threshold</code> ●	float	정책에 따름	AA-only 적용 SOC 임계값 직접 지정

`--soc-stacking-policy` 선택지:

값	AA-only 시작 SOC	근거
none	없음 (모든 SOC에 전체 시퀀스)	정책 없음
conservative ✓	$SOC \geq 0.50$	보수적 마진
literature	$SOC \geq 0.40$	Kim et al. 논문 Fig. 4c 기준

5.5 Li 사이트 확장 (GA 및 GA supplement)

옵션	타입	기본값	설명
<code>--ga-expand-li-sites</code> ✓	flag	on	GA Li 후보 사이트를 흑연 헥사곤 중심으로 확장
<code>--no-ga-expand-li-sites</code>	flag	—	입력 구조의 Li 사이트만 사용
<code>--hollow-site-tol</code> ●	float	0.35 Å	빈 사이트 중복 제거 허용 거리
<code>--cc-bond-cutoff</code> ●	float	1.85 Å	C-C 결합 판별 기준거리 (헥사곤 감지용)

Full enumeration은 항상 원래 N_LI_TOTAL 사이트만 사용합니다.

Expanded hollow site 탐색은 GA (`--ga`) 및 GA supplement (`--write-training-cfg`) 경로에만 적용됩니다.

5.6 모티프 시딩 (Motif-Seeded GA)

고SOC에서 물리적으로 타당한 초기 population을 제공하여 수렴 속도를 향상시킵니다.

옵션	타입	기본값	설명
<code>--ga-motif-seeding</code> ●	choice	<code>high-soc</code>	모티프 시딩 모드
<code>--motif-seed-soc-threshold</code> ●	float	<code>0.50</code>	high-soc 모드 발동 SOC 임계값
<code>--motif-rh-fraction</code> ●	float	<code>0.70</code>	RH-like/균일 분포 시드 비율
<code>--motif-facing-fraction</code> ●	float	<code>0.20</code>	Li-Li facing pair 시드 비율
<code>--motif-random-fraction</code> ●	float	<code>0.10</code>	무작위 시드 비율 (세 비율의 합이 1이 되도록 정규화)
<code>--facing-mutation-prob</code> ●	float	<code>0.20</code>	facing 돌연변이 비율 (새 개체 중)
<code>--ga-uniform-vacancy-mode</code> ●	choice	<code>seed</code>	균일 vacancy 분포 강제 방식
<code>--uniform-vacancy-tolerance</code> ●	int	<code>1</code>	레이어간 vacancy 수 최대 허용 편차

`--motif-seed-soc-threshold` 설정 지침

고SOC에서는 Li-Li 반발이 지배적이라 균일 분포(RH-like) 구조가 안정합니다. 이 임계값 이상의 SOC에서 모티프 시딩이 발동합니다.

값	연계 정책	추천 상황
<code>0.40</code>	<code>--soc-stacking-policy literature</code>	논문 기준에 맞춘 일관성
<code>0.50</code>	<code>--soc-stacking-policy conservative</code>  기본값	보수적 마진, 대부분의 경우
<code>0.60 - 0.75</code>	—	중간 SOC 수렴이 충분하고 고SOC만 보강할 때

권장: `--soc-stacking-policy` 값과 일치시키세요. `conservative` 사용 시 `0.50`, `literature` 사용 시 `0.40`.

5.7 중복 제거 (Deduplication)

옵션	타입	기본값	설명
<code>--dedup-mode</code> ●	choice	<code>symmetry</code>	중복 구조 필터링 방식
<code>--dedup-ltol</code> ●	float	<code>0.2</code>	StructureMatcher 격자 길이 허용오차
<code>--dedup-stol</code> ●	float	<code>0.3</code>	StructureMatcher 사이트 허용오차
<code>--dedup-angle-tol</code> ●	float	<code>5.0</code> °	StructureMatcher 각도 허용오차

--dedup-mode 선택지:

값	설명
<code>symmetry</code>	pymatgen StructureMatcher로 대칭 동치 제거
<code>exact</code>	정확히 동일한 분수좌표 집합만 제거
<code>none</code>	중복 제거 비활성화

소스 내 dedup: full_enum, ga_supplement, variant_structures 각각에 독립적으로 적용됩니다.

Cross-source dedup (`--write-training-cfg` 전용): 세 소스를 합쳐 소스 우선순위 순으로 StructureMatcher 비교를 수행합니다. 우선순위: `full_enum` > `variant_structures` > `ga_supplement`. 중복 시 우선순위가 높은 소스의 구조가 남습니다. 실행 완료 후 SOC별 소스 카운트 테이블이 출력됩니다.

5.8 에너지 백엔드 (MTP / Ewald)

옵션	타입	기본값	설명
<code>--energy-backend</code>	choice	<code>mtp-relax</code>	에너지 평가 방법
<code>--mlp-bin</code>	str	<code>mlp</code>	MLIP 실행파일 경로 또는 이름
<code>--mlip-ini</code>	str	<code>mlip.ini</code>	<code>mlp relax</code> 에 사용할 ini 파일
<code>--mtp-force-tolerance</code>	float	없음	<code>--force-tolerance</code> 값 (미지정 시 mlp 기본값)
<code>--mtp-workers</code>	int	<code>1</code>	동시 <code>mlp relax</code> 프로세스 수
<code>--mtp-batch-size</code>	int	<code>0</code>	CFG 청크 크기 (0=SOC/세대 전체를 하나로)
<code>--mtp-keep-workdirs</code>	flag	off	임시 CFG/로그 보존 (디버깅용)
<code>--mtp-failed-energy</code>	float	<code>1e100</code>	완화 실패 시 패널티 에너지
<code>--mtp-extra-args</code>	str*	없음	<code>mlp relax</code> 에 추가할 인자들

5.9 I/O 최적화 옵션

GA에서 세대마다 대량으로 생성되던 파일을 줄이는 옵션들입니다.

옵션	기본값	설명
<code>--mtp-write-aggregate</code>	on	세대별 <code>IN_*.cfg</code> 집계 파일 쓰기
<code>--no-mtp-write-aggregate</code>	—	<code>IN_*.cfg</code> 생략 (대폭 I/O 절감)

옵션	기본값	설명
<code>--mtp-single-log-per-soc</code>	on	SOC별 단일 로그 파일
<code>--no-mtp-single-log-per-soc</code>	—	chunk별 개별 로그
<code>--mtp-flush-relaxed-per-soc</code>	on	SOC 완료 후 relaxed CFG 통합 1회 쓰기
<code>--no-mtp-flush-relaxed-per-soc</code>	—	세대마다 즉시 쓰기

5.10 GA 고급 옵션

조기 종료 (Early Stopping)

옵션	타입	기본값	설명
<code>--ga-early-stop-gens</code>	int	0 (꺼짐)	N 세대 연속 개선 없으면 GA 조기 종료

적응적 돌연변이 (Adaptive Mutation)

옵션	타입	기본값	설명
<code>--ga-adaptive-mutation</code>	flag	on	정체 시 random/fullwipe 비율 자동 증가
<code>--no-ga-adaptive-mutation</code>	flag	—	적응적 돌연변이 비활성화
<code>--ga-adaptive-after-gens</code>	int	5	몇 세대 정체 후 adaptive 발동할지
<code>--ga-fullwipe-fraction</code>	float	0.05	fullwipe(전체 Li 재생성) 비율 (정체 시)

다양성 필터 (Diversity Filter)

옵션	타입	기본값	설명
<code>--ga-min-symdiff</code>	int	0 (꺼짐)	새 개체와 현 세대 멤버 간 최소 Hamming 거리

Elite 섭동 (Elite Perturbation)

옵션	타입	기본값	설명
<code>--ga-elite-perturb-fraction</code>	float	0.05	최우수 구조 1-2 사이트 교체 비율

5.11 ★ 학습 데이터 생성 옵션 (v6 신규)

write-training-cfg 모드 (권장)

`--ga` 없이 전수열거 결과를 학습 데이터로 수집하고, 저SOC에서는 GA supplement로 expanded site 구조를 추가합니다.

옵션	타입	기본값	설명
<code>--write-training-cfg</code> ★	flag	off	학습 데이터 CFG 생성 모드 활성화
<code>--training-data-soc-threshold</code> ★	float	0.50	이 값 미만 SOC에서 GA supplement 실행. 1.01 권장 (모든 SOC 대상)
<code>--training-data-top-n</code> ★	int	20	SOC당 수집할 최대 구조 수 (full_enum, ga_supplement, variant 각각 적용). 0=제한 없음
<code>--training-data-cfg</code> ★	str	training_data.cfg	통합 CFG 출력 파일명
<code>--training-data-cif</code> ✓ ★	flag	on	개별 CIF 파일 쓰기
<code>--no-training-data-cif</code> ★	flag	—	CIF 생략 (CFG만 생성, 빠름)

training-data 모드 (레거시)

`--training-data` 사용 시: $SOC < threshold$ 는 GA, $SOC \geq threshold$ 는 전수열거로 수집합니다. `--write-training-cfg` 가 더 권장됩니다.

옵션	타입	기본값	설명
<code>--training-data</code> ★	flag	off	학습 데이터 수집 모드 (--ga 와 함께 사용)

5.12 ★ 구조 Variant 생성 옵션 (v6 신규)

10가지 설계 원칙으로 체계적인 Li 배치를 생성합니다. `--write-training-cfg` 시 `variant_structures/` 서브디렉토리에 저장되며, `--ga` 시 초기 population seed로 주입됩니다.

옵션	타입	기본값	설명
<code>--structured-variants</code> ✓ ★	flag	on	Variant 구조 생성 활성화
<code>--no-structured-variants</code> ★	flag	—	Variant 생성 비활성화

Variant 종류:

Variant	전략	물리적 의미
6. Top-Biased Removal	높은 z-layer부터 우선 제거	표면층 방출 구조
7. Cluster-Preserving	가장 많은 이웃을 가진 Li 유지	군집 안정 구조
8. Interlayer Skipping	짝수 레이어만 Li 배치	다층 간 상호작용 감소
9. Density Gradient	low-z → high-z 방향 밀도 증가	전위 구배 모사
10. Central Core Removal	XY 중심부 Li 제거, 외곽 유지	가장자리 우선 삽입
11. Random Keep	무작위 n_li_keep 선택	단순 무작위 sampling
12a. Stripe X	낮은 x 방향 줄무늬 유지	도메인 패턴
12b. Stripe Y	낮은 y 방향 줄무늬 유지	도메인 패턴
13. Grid-patterned	2×2 XY 사분면 균등 분포	격자 정규성
14. Layer Compression	하반부 레이어만 Li 배치	압축된 계면
15. Low-Distance Repulsion	가까운 Li-Li 쌍 우선 제거	최소 거리 만족 구조

5.13 출력 제어

옵션	타입	기본값	설명
<code>--no-poscar</code> 	flag	off	POSCAR 파일 생략 (전수 열거 모드)
<code>--sig-figs</code> 	int	8	POSCAR 좌표 유효 자릿수

6. 사용 예시

예시 1: 최소 실행 (Ewald 에너지, 전수 열거)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \
  --structure LiC6.cif \
  --energy-backend ewald
```

예시 2: GA 기본 실행 (Ewald 에너지)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --energy-backend ewald \  
  --ga \  
  --ga-pop 120 \  
  --ga-gens 60
```

예시 3: ★ MTP 학습 데이터 생성 (권장)

핵심 사용 사례. 전수열거 구조 + expanded hollow site GA supplement + Variant 구조를 모든 SOC에서 수집합니다.

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --ga-min-li-li-same-layer 4.00 \  
  --ga-pop 100 \  
  --ga-gens 50 \  
  --ga-expand-li-sites \  
  --stacking-sequences ALL \  
  --soc-stacking-policy conservative \  
  --supercell 2 2 4 \  
  --write-training-cfg \  
  --training-data-soc-threshold 1.01 \  
  --training-data-top-n 30 \  
  --structured-variants \  
  --energy-backend ewald
```

생성 결과:

li_configs/training_data.cfg — 모든 학습 구조 통합 CFG

li_configs/training_data/SOC_*/full_enum/ — 전수열거 대칭유일 구조 (최대 30개/SOC)

li_configs/training_data/SOC_*/ga_supplement/ — hollow site GA 구조 (최대 30개/SOC)

li_configs/training_data/SOC_*/variant_structures/ — Variant 설계 구조 (sym-dedup 후)

예시 4: MTP 학습 데이터 — CIF 없이 빠르게

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --supercell 2 2 4 \  
  --stacking-sequences ALL \  
  --soc-stacking-policy conservative \  
  --write-training-cfg \  
  --training-data-soc-threshold 1.01 \  
  --training-data-top-n 20 \  
  --no-training-data-cif \  
  --energy-backend ewald
```

CIF 파일 생략 → 디스크 절약, 속도 향상

training_data.cfg 만 생성

예시 5: GA + Variant seeding ★

Variant 구조가 GA 초기 population에 자동 주입됩니다.

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --energy-backend ewald \  
  --ga \  
  --ga-pop 150 \  
  --ga-gens 80 \  
  --stacking-sequences ALL \  
  --soc-stacking-policy conservative \  
  --ga-expand-li-sites \  
  --structured-variants \  
  --ga-motif-seeding high-soc \  
  --supercell 2 2 4
```

예시 6: MTP 에너지 GA (논문 재현)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --mlip-ini mlip.ini \  
  --ga \  
  --stacking-sequences ALL \  
  --soc-stacking-policy literature \  
  --ga-pop 150 \  
  --ga-gens 100 \  
  --ga-motif-seeding always \  
  --no-mtp-write-aggregate \  
  --output ./results_mtp_literature
```

예시 7: MTP 병렬 평가

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --mlip-ini mlip.ini \  
  --ga \  
  --mtp-workers 4 \  
  --mtp-batch-size 30 \  
  --output ./results_mtp_parallel
```

예시 8: 대규모 계산 (I/O 절감 + 조기종료)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --mlip-ini mlip.ini \  
  --ga \  
  --stacking-sequences ALL \  
  --soc-stacking-policy literature \  
  --ga-pop 150 \  
  --ga-gens 80 \  
  --ga-early-stop-gens 20 \  
  --ga-min-symdiff 2 \  
  --ga-fullwipe-fraction 0.08 \  
  --ga-elite-perturb-fraction 0.08 \  
  --no-mtp-write-aggregate \  
  --output ./results_large
```

예시 9: 특정 슈퍼셀 + 특정 스택킹 + 특정 SOC

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --supercell 3 3 2 \  
  --stacking-sequences AAAA ABAA ABAB \  
  --soc-levels 18 24 30 36 \  
  --energy-backend ewald \  
  --output ./results_3x3x2
```


예시 10: 디버그 실행

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --mlip-ini mlip.ini \  
  --ga \  
  --ga-gens 10 \  
  --ga-pop 30 \  
  --mtp-keep-workdirs \  
  --soc-levels 4 8 \  
  --output ./debug_run
```

7. 워크플로우 가이드

탐색 규모별 추천 설정

소형 테스트 (빠른 확인)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --energy-backend ewald \  
  --supercell 2 2 2 \  
  --stacking-sequences AA \  
  --write-training-cfg \  
  --training-data-soc-threshold 1.01 \  
  --training-data-top-n 5 \  
  --no-ga-expand-li-sites \  
  --output ./test_run
```

중형 학습 데이터 (2×2×4, Ewald)

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --energy-backend ewald \  
  --supercell 2 2 4 \  
  --stacking-sequences ALL \  
  --soc-stacking-policy conservative \  
  --write-training-cfg \  
  --training-data-soc-threshold 1.01 \  
  --training-data-top-n 30 \  
  --ga-pop 100 --ga-gens 50 \  
  --structured-variants \  
  --output ./medium_training
```

대형 MTP 학습 데이터 (권장 최종 워크플로우)

Step 1: Ewald로 빠른 구조 선별 → training_data.cfg 생성

```
python enumerate_li_configs_Final_optimized_motif_v6_mtp_train.py \  
  --structure LiC6.cif \  
  --energy-backend ewald \  
  --supercell 2 2 4 \  
  --stacking-sequences ALL \  
  --soc-stacking-policy conservative \  
  --write-training-cfg \  
  --training-data-soc-threshold 1.01 \  
  --training-data-top-n 30 \  
  --ga-pop 150 --ga-gens 80 \  
  --structured-variants \  
  --output ./ewald_training
```

Step 2: training_data.cfg를 MTP 학습에 활용

(DFT 재계산 또는 직접 MTP 학습)

학습 데이터 구성 전략

구조 소스	특징	역할	Cross-dedup 우선순위
full_enum	원래 Li site, 대칭 유일, 저에너지 우선	안정 구조 기준	① 최우선 (항상 생존)
variant_structures	물리적 설계 원칙 기반, 원래 Li site 사용	체계적 기하 다양성	② full_enum 중복 제외 후 생존

구조 소스	특징	역할	Cross-dedup 우선순위
<code>ga_supplement</code>	hollow site 포함 expanded space, GA 탐색	hollow-center 신규 구조	③ 앞 두 소스에 없는 구조만 추가

소형 슈퍼셀 (2×2×2)에서 **variant=0**은 정상입니다. full_enum이 원래 Li site 전체 조합을 열거하므로 같은 사이트를 사용하는 variant 구조는 항상 중복입니다. 2×2×4 이상의 대형 슈퍼셀에서는 full_enum이 top-N만 선택하므로 variant가 genuinely unique한 구조를 제공합니다.

`--training-data-soc-threshold 1.01`: 모든 SOC 레벨(SOC 0 ~ 1.0)에 `ga_supplement`와 `variant_structures`를 적용합니다. 고SOC에서도 다양한 훈련 구조를 확보하려면 이 값을 사용하세요.

SOC-의존 스택킹 정책 선택 가이드

SOC 구간	물리적 상태	권장 정책
SOC < 0.20	C-C dominant	none (AB 허용)
SOC < 0.40	Li-C competitive	none 또는 conservative
SOC ≥ 0.40	Li-Li dominant	literature (AA-only)
SOC ≥ 0.50	AA 완전 지배	conservative (기본값)

8. 성능 튜닝 가이드

GA 수렴 속도 향상

상황	권장 조치
수렴이 너무 느림	<code>--ga-early-stop-gens 15</code> 설정
조기 수렴 의심	<code>--ga-min-symdiff 2</code> 또는 3 설정
국소 최소에 갇힘	<code>--ga-fullwipe-fraction 0.10</code> 증가
미세 탐색 강화	<code>--ga-elite-perturb-fraction 0.10</code> 증가
고SOC 수렴 불량	<code>--ga-motif-seeding always</code> + <code>--motif-rh-fraction 0.80</code>

학습 데이터 생성 성능

상황	권장 조치
dedup이 너무 느림 (대형 슈퍼셀)	<code>--training-data-top-n 30</code> (기본 20보다 약간 높게 설정)
dedup 속도 더 향상	<code>--dedup-mode exact</code> (symmetry 대신 좌표 해시만 사용)
CIF 불필요	<code>--no-training-data-cif</code>
ga_supplement 생략	<code>--no-ga-expand-li-sites</code>
variant 생략	<code>--no-structured-variants</code>

MTP I/O 최적화

상황	권장 조치
디스크 공간 부족	<code>--no-mtp-write-aggregate</code>
mlp 자체가 멀티코어	<code>--mtp-workers 1</code> (기본값, 충돌 방지)
mlp 싱글코어 + 여유 코어 있음	<code>--mtp-workers 4 --mtp-batch-size 30</code>

9. 출력 파일 설명

실행 중 로그 형식

`--write-training-cfg` 모드에서 각 소스별 진행 상황이 고정 너비 태그로 구분되어 출력됩니다:

```
[full_enum    ] SOC=0.2500  6 candidates → 3 sym-unique
[ga_supplement] SOC=0.2500  GA 시작 (pop=100, gens=50, 24 Li sites, top-30) ...
[ga_supplement] SOC=0.2500  204 evaluated → 6 sym-unique
[variant_struct] SOC=0.2500  8 variants × 2 stackings = 16 → 2 sym-unique
[cross-dedup   ] SOC=0.2500  11 → 9 unique (2 cross-source duplicates removed)
>>> SOC=0.2500 nLi= 2  full_enum= 3  ga_supplement= 6  variants= 0  total= 9
```

태그	소스	설명
<code>[full_enum]</code>	전수열거	원래 Li site 대칭유일 구조 수
<code>[ga_supplement]</code>	GA supplement	GA 시작 파라미터 및 평가 결과
<code>[variant_struct]</code>	Variant 구조	생성 공식 <code>N variants × M stackings = K → J sym-unique</code>
<code>[cross-dedup]</code>	Cross-source dedup	중복 제거 전후 구조 수 (중복 없을 시 생략)

태그	소스	설명
>>> soc=...	최종 집계	SOC별 소스별 최종 구조 수 한줄 요약

enumeration_summary.json

전체 실행의 JSON 요약 파일. v6 신규 필드 포함:

```

{
  "structure_file": "LiC6.cif",
  "supercell": [2, 2, 4],
  "ga_enabled": false,
  "energy_backend": "ewald",
  "stable_by_soc": [...],
  "soc_levels": [
    {
      "n_Li": 8,
      "SOC": 0.5,
      "method": "sym",
      "n_ga_variant_seeds_injected": 6,
      "top_stable": [...]
    }
  ],
  "training_data": {
    "mode": "write_training_cfg",
    "n_total_structures": 284,
    "cfg_file": "li_configs/training_data.cfg",
    "cif_written": true,
    "soc_breakdown": [
      {
        "SOC": 0.5,
        "n_Li": 8,
        "method": "full_enum",
        "n_full_enum": 6,
        "n_training_structures": 6,
        "n_ga_supplement": 5,
        "n_variant_structures": 3
      }
    ]
  }
}

```

실행 완료 시 자동 출력되는 summary table

Training data summary (after cross-source dedup; priority: full_enum > variants > ga_supplement):

SOC	n_Li	full_enum	ga_suppl	variants	total
0.0000	0	2	0	0	2
0.2500	2	6	6	0	12
0.5000	4	6	15	3	24
...					
Total		30	58	5	93

`full_enum`: cross-dedup 후 생존한 전수열거 구조 수

`ga_suppl`: `full_enum`·`variants`와 중복되지 않는 `ga_supplement` 구조 수

`variants`: `full_enum`과 중복되지 않는 variant 구조 수 (소형 슈퍼셀에서는 0)

참고: 각 SOC의 `>>> SOC=...` 줄은 CFG 쓰기 직전 출력되고, 최종 summary table은 모든 SOC 처리 완료 후 출력됩니다.

training_data.cfg

MTP 학습에 바로 사용 가능한 통합 CFG 파일. 모든 SOC의 `full_enum` + `ga_supplement` + `variant_structures` 구조를 cross-source dedup 후 포함합니다.

CIF 파일 (`training_data/SOC_*/`)

`full_enum/`: 원래 Li site 대칭유일 구조

`ga_supplement/`: hollow site 확장 공간에서 GA 탐색한 sym-unique 구조

`variant_structures/`: 10가지 설계 원칙으로 생성된 sym-unique 구조

파일명 형식: `rank00001_{STACKING}_nLi{N}_SOC{X}_{E}eV.cif`

GA 수렴 그래프 (`ga_convergence_nLi{N}.png`)

세대별 에너지 분포 시각화. v6에서 `variant_seed` 타입이 추가됩니다.

10. 자주 묻는 질문

Q. `--write-training-cfg` 와 `--training-data` 의 차이는?

A. `--write-training-cfg` 는 `--ga` 없이 전수열거 결과를 학습 데이터로 수집합니다 (권장). `--training-data` 는 레거시 모드로 `--ga` 와 함께 사용합니다.

Q. `--training-data-soc-threshold` 를 왜 `1.01` 로 설정하나요?

A. 기본값 `0.50` 은 $SOC \geq 0.50$ 에서 `ga_supplement`를 건너뛸니다. `1.01`로 설정하면 모든 SOC (최대 1.0)에서 `ga_supplement`와 `variant_structures`가 생성되어 학습 데이터 다양성이 향상됩니다.

Q. `full_enum`과 `ga_supplement`가 같은 구조를 생성하지 않나요?

A. 생성할 수 있습니다. `ga_li_frac` 는 원래 `li_frac` 사이트를 포함하므로, GA가 충분히 탐색하면 `full_enum`과 동일한 원자 위치의 구조를 발견합니다. 이를 위해 **cross-source dedup**이 적용됩니다: 소스 우선순위(`full_enum` → `variant_structures` → `ga_supplement`) 순으로 StructureMatcher 비교를 수행하여 중복을 제거합니다. `full_enum` 구조는 항상 생존하고, `ga_supplement`는 hollow-center 사이트에서만 발견되는 신규 구조만 추가합니다.

Q. Variant 구조 수가 0으로 표시됩니다.

A. 소형 슈퍼셀($2 \times 2 \times 2$)에서는 정상입니다. Variant는 원래 `li_frac` 사이트(8개)를 사용하는데, `full_enum`이 이 사이트들의 모든 sym-unique 조합을 열거합니다. 따라서 variant가 새로운 구조를 추가할 수 없습니다. $2 \times 2 \times 4$ 이상의 슈퍼셀에서는 `full_enum`이 에너지 상위 top-N만 선택하므로, variant가 선택되지 않은 기하학적 패턴 구조를 제공합니다.

Q. Variant 구조가 매우 적게 생성됩니다 (예: 1~2개).

A. 극단적인 SOC(0 또는 1에 가까움)에서는 기하학적으로 distinct한 variant 수가 제한됩니다. 이는 정상 동작입니다.

Q. GA에서 mutation(변형)은 어떤 구조에 적용되나요?

A. `vacancy_swap`, `layer_swap`, `stacking_flip`, `elite_perturb` 변형은 생존자(에너지 임계값 내 구조)를 부모로 삼아 적용됩니다. 생존자가 없을 경우 전체 개체를 재생성합니다. 추가로 `random` (~25%)과 `fullwipe` (~5%) 유형은 부모와 무관하게 독립 생성됩니다. 따라서 GA는 좋은 구조를 보존하면서 탐색 공간을 확장합니다.

Q. `--ga-threshold-ev` 를 얼마로 설정해야 하나요?

A. 기본값 `0.5 eV` 가 대부분의 경우에 적합합니다. 이 값은 `min_E + threshold` 이내 구조가 생존하는 윈도우입니다. 대형 슈퍼셀(Li 수 많음)에서 수렴이 느리면 `0.3 eV` 로 낮춰 선택 압력을 강화하세요. 소형 슈퍼셀에서 다양성이 부족하면 `0.7 - 1.0 eV` 로 높이세요.

Q. `--motif-seed-soc-threshold` 를 얼마로 설정해야 하나요?

A. `--soc-stacking-policy` 와 맞추는 것이 권장됩니다: `conservative` 정책 사용 시 `0.50` (기본값), `literature` 정책 사용 시 `0.40`. 이 SOC 이상에서 Li-Li 반발이 지배적이 되어 균일 분포(RH-like) 시딩이 효과적입니다.

Q. `NoValidGACandidateError` 가 발생합니다.

A. `--ga-min-li-li-same-layer` 값을 낮추거나 (예: 3.5), `--no-ga-expand-li-sites` 로 확장 사이트를 끄세요.

Q. GA와 전수 열거 중 무엇을 써야 하나요?

A. Li 수가 많아 $C(N,k) > 10^6$ 이면 GA 권장. `--write-training-cfg` 모드에서는 전수열거로 안정 구조를 확인하고, GA supplement로 expanded space를 추가 탐색합니다.

Q. 디스크 사용량이 너무 큼니다.

A. `--no-training-data-cif` 로 CIF 생략, `--no-mtp-write-aggregate` 로 `IN_*.cfg` 생략, `--training-data-top-n 10` 으로 구조 수를 줄이세요.

마지막 수정: 2026-05-17 | 스크립트 버전: v6_mtp_train